

hello

safe memory access according to cyrille

- memory should have been allocated (statically)
- it should be initialized (can't read undefined data)
- you are only allowed to free once (and you aren't allowed to use freed memory)
- no memory exhaustion

cyrille talks about the Java GC with apache zookeeper, where apparently the GC took so long that all of the sessions timed out, causing them to try to reconnect, which of course takes up memory, which of course starts the GC again (ad nauseum).

cyrille says that the garbage collector of course can't clean up data which you have a reference to (which the GC obviously doesn't collect; don't remove if in use).

common problems

- unwanted aliasing, race conditions
- buffer overflows, UAF
- null pointer deref
- double free, memory leak

common issues

- wow silly RCE oopsie
- memory leak makes computer explode -> DoS

also like another note about GC -> cyclic references can't be removed with reference counting.

note from myself: there does exist multiple GC techniques to make this smooth.

- like i dunno combine mark & sweep with generational GC or incremental GC
- probably a handful of papers just about this.

Valgrind

it's time

actually it isn't

fuzzing

fuzzing usually only detects when the program crashes. I think

how does fuzzing works.

- generate slightly invalid inputs based on valid inputs
 - to test branches of execution. Example of not having a valid PDF magic string being useless is brought up

Other half of lecture

Vulnerability detection of known vulnerabilities. -> DevSecOps / AntiVirus detect unauthorized changes with checksums.

Firewalls and intrusion detection (apparently what?) wow I love snort and fail2ban or AI

Security Information and event management (big letters to say grafana + other)

Forensics wow!

- It's like normal forensics, but with hardware
- making static images, but they have to be able to be used in court (unaltered). Could e.g a blocker, where the write signal

is being blocked (no writing Wow!)